



Lab 11 – Worksheet

Name: Ghazi Hussain, Sarosh Mehdi, Sualeh Abbas, Hamza Hassan	ID: Group 10	Section: T2
---	--------------	-------------

Task a: vga_sync

Code: Design module vga_sync

Provide appropriately commented code for designed module, the code should contain meaningful variable naming.

*Add snippet or Copy paste the code written in the Editor window. Make sure the irrelevant area of the snip is cropped.

CODE FOR VGA_SYNC

```
1  `timescale 1ns / 1ps
2
3  module vga_sync (
4      input [9:0] h_count,
5      input [9:0] v_count,
6      output h_sync,
7      output v_sync,
8      output video_on, // active area
9      output [9:0] x_loc, // current pixel x- location
10     output [9:0] y_loc // current pixel y-location
11 );
12 // horizontal
13 localparam HD = 640; // Horizontal Display Area
14 localparam HF = 16; // Horizontal (Front Porch) Right Border
15 localparam HB = 48; // Horizontal (Back Porch) Left Border
16
17 localparam HR = 96; // Horizontal Retrace
18
19 // vertical
20 localparam VD = 480; // Vertical Display Area
21 localparam VF = 10; // Vertical (Front Porch) Bottom Border
22 localparam VB = 33; // Vertical (Back Porch) Top Border
23 localparam VR = 2; // Vertical Retrace
24
25 assign x_loc = h_count;
26 assign y_loc = v_count;
27 // Generate video_on signal (inside display region)
28 assign video_on = (h_count < HD) && (v_count < VD);
29 // Generate h_sync signal (active low)
30 assign h_sync = (h_count < (HD + HF)) || (h_count >= (HD + HF + HR)) ? 1'b1 : 1'b0;
31 // Generate v_sync signal (active low)
32 assign v_sync = (v_count < (VD + VF)) || (v_count >= (VD + VF + VR)) ? 1'b1 : 1'b0;
33 endmodule
```

TESTBENCH

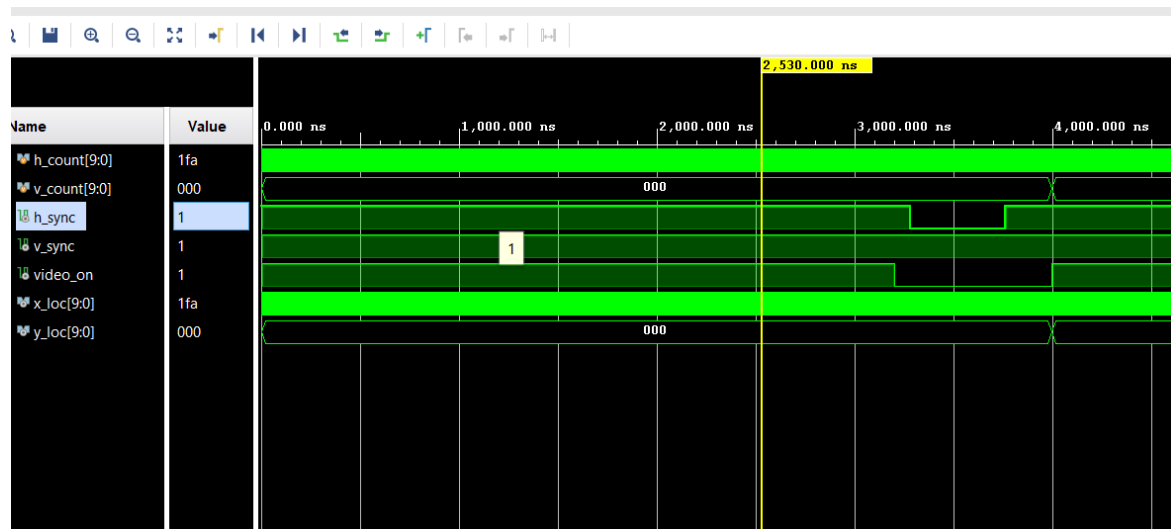
```
1  `timescale 1ns / 1ps
2
3  module vga_sync_tb;
4      reg [9:0] h_count, v_count;
5      wire h_sync, v_sync, video_on;
6      wire [9:0] x_loc, y_loc;
7
8      vga_sync uut (h_count, v_count, h_sync, v_sync, video_on, x_loc, y_loc);
9
10     initial begin
11         h_count = 0;
12         v_count = 0;
13         repeat (525) begin
14             repeat (800) begin
15                 #5 h_count = h_count + 1;
16                 if (h_count == 799) begin
17                     h_count = 0;
18                     v_count = v_count + 1;
19                     if (v_count == 524) v_count = 0;
20                 end
21             end
22         end
23         $stop;
24     end
25 endmodule
```

Results (Waveforms)

**Add snip of relevant signals' waveforms. Make sure the irrelevant area of the snip is cropped. make sure to scale properly for visibility of all case.*

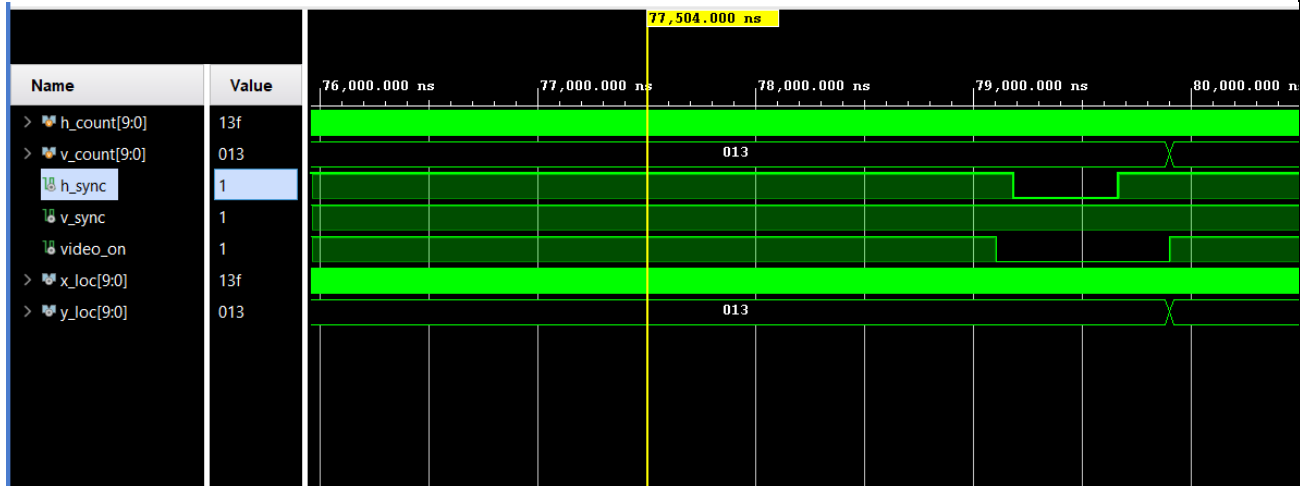
TEST CASE 1:-

The simulation is showing the **very first line** of the video frame ($v_count = 0$). The current pixel being displayed is the **506th pixel** in the horizontal scan line ($h_count = 506$). Since $video_on = 1$, this pixel **will be shown on the screen**. Both h_sync and v_sync are **high**, meaning we're in the sync pulse phase — possibly at the end of a line and also during the vertical retrace.



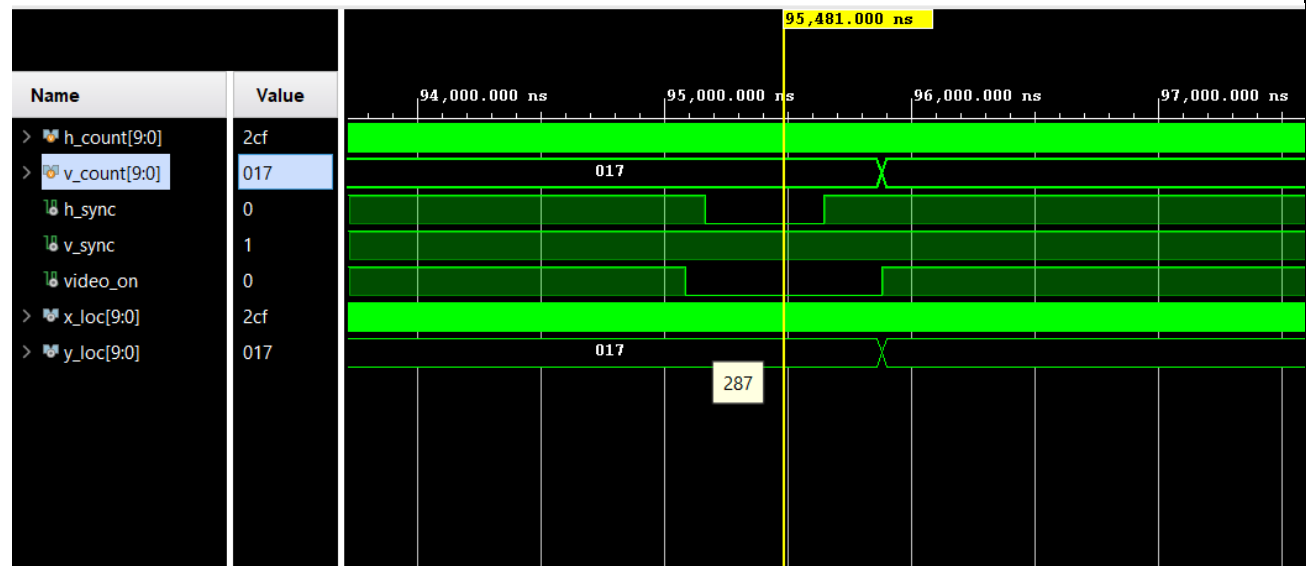
TEST CASE 2”:-

- This is the **320th pixel of the 20th line** of the frame. The pixel is in the **visible region** (`video_on` = 1), so it's part of what the user would see on a display. The **horizontal sync** (`h_sync`) and **vertical sync** (`v_sync`) are both high — which typically indicates the system is in a sync interval, but in this case, it's happening while `video_on` is still high. Transition to the next line (`v_count` = 014) is about to happen, as shown at the next major timestamp (~80,000 ns).

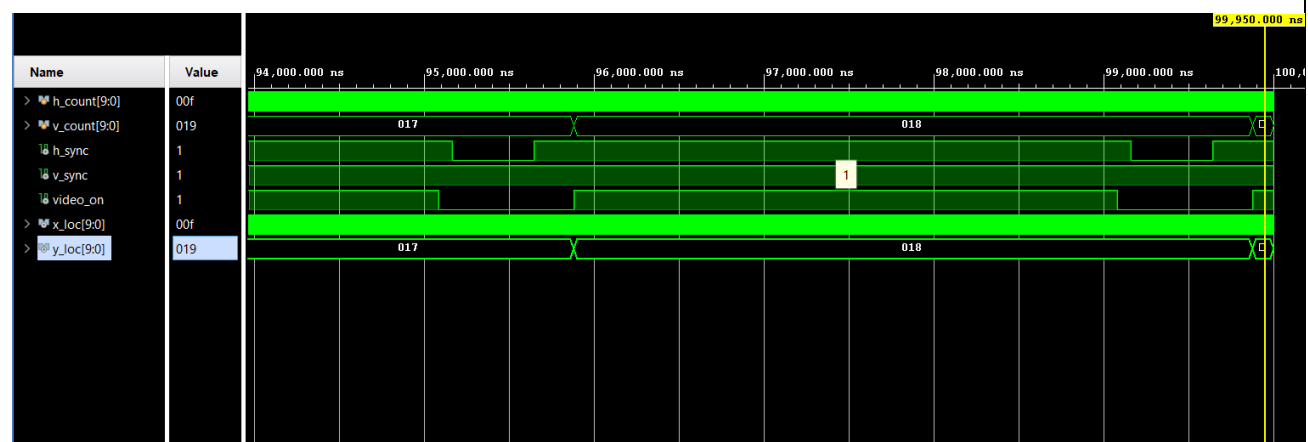


TEST CASE 3:-

- **Pixel Location:** You're at pixel (719, 23), which is typically near the **end of a horizontal line** (since horizontal counts usually wrap at 800 in 640x480 mode). **video_on = 0**; **h_sync = 0**: No horizontal sync pulse at this moment — so this is inside the visible or front porch period. **v_sync = 1**; **vertical sync pulse**, which means this is during a frame refresh (not visible to screen). Even though **x_loc** and **y_loc** give pixel coordinates, **no image is shown** since **video_on** is 0



TEST CASE 4:-



Comments

**Observation/Comments on the obtained results/working of code.*

This module is designed to **generate VGA synchronization signals** (`h_sync`, `v_sync`) and **identify the active video display region** using `video_on`, `x_loc`, and `y_loc`. It uses input counters `h_count` and `v_count` which are typically driven by a VGA timing generator elsewhere in your system.

Task b: Pixel Generation

Code: Design pixel_gen

Provide appropriately commented code for designed module and the testbench, the, code should contain meaningful variable naming.

**Add snippet or Copy paste the code written in the Editor window. Make sure the irrelevant area of the snip is cropped.*

CODE FOR DISPLAYING FLAG ON SCREEN:-

```

1  `timescale 1ns / 1ps
2  module pixel_gen(
3      input clk_d,
4      input [9:0] pixel_x,    // Horizontal pixel coordinate (0-639)
5      input [9:0] pixel_y,    // Vertical pixel coordinate (0-479)
6      input video_on,         // Video signal active
7      output reg [3:0] red = 0, // 4-bit Red component
8      output reg [3:0] green = 0, // 4-bit Green component
9      output reg [3:0] blue = 0 // 4-bit Blue component
10 );
11
12 // Screen dimensions (based on testbench usage)
13 localparam SCREEN_WIDTH = 640;
14 localparam SCREEN_HEIGHT = 480;
15 localparam MID_Y = SCREEN_HEIGHT / 2; // Midpoint for vertical split (240)
16
17 always @(posedge clk_d) begin
18     if (video_on) begin
19         // Check vertical position to determine color
20         if (pixel_y < MID_Y) begin
21             // Top half: Green
22             red <= 4'h0; // No Red
23             green <= 4'hF; // Full Green
24             blue <= 4'h0; // No Blue
25         end
26     else begin
27         // Bottom half: Red (pixel_y >= MID_Y)
28         red <= 4'hF; // Full Red
29         green <= 4'h0; // No Green
30         blue <= 4'h0; // No Blue
31     end
32 end

```

TEST BENCH:-

```
`timescale 1ns / 1ps

module pixel_gen_full_screen_tb;

    reg clk_d;
    reg [9:0] pixel_x;
    reg [9:0] pixel_y;
    reg video_on;
    wire [3:0] red;
    wire [3:0] green;
    wire [3:0] blue;

    integer x, y;

    // Instantiate the Device Under Test (DUT)
    pixel_gen uut (
        .clk_d(clk_d),
        .pixel_x(pixel_x),
        .pixel_y(pixel_y),
        .video_on(video_on),
        .red(red),
        .green(green),
        .blue(blue)
    );

    // Clock generator: 10 ns period (100 MHz)
    initial begin
        clk_d = 0;
        forever #5 clk_d = ~clk_d;
    end

    // Full-screen scan simulation
    initial begin
```

```

32 initial begin
33     video_on = 1; // Enable video output
34
35     // Iterate through all pixels on a 640x480 screen
36     for (y = 0; y < 480; y = y + 1) begin
37         for (x = 0; x < 640; x = x + 1) begin
38             // Apply current pixel coordinates
39             pixel_x = x;
40             pixel_y = y;
41
42             // Wait for the rising edge of the clock for outputs to update
43             @(posedge clk_d);
44
45             // Optional: Display color values for verification (can generate large output)
46             // Use conditional display or file logging for large simulations
47             // if (x == 0) // Example: Display only first pixel of each line
48             //     $display("At (x=%0d, y=%0d): R=%h, G=%h, B=%h", x, y, red, green, blue);
49
50         end
51     end
52
53     // Add a small delay before stopping simulation
54     #100;
55     $display("Simulation finished after scanning full screen.");
56     $stop; // Stop the simulation
57 end
58
59 endmodule

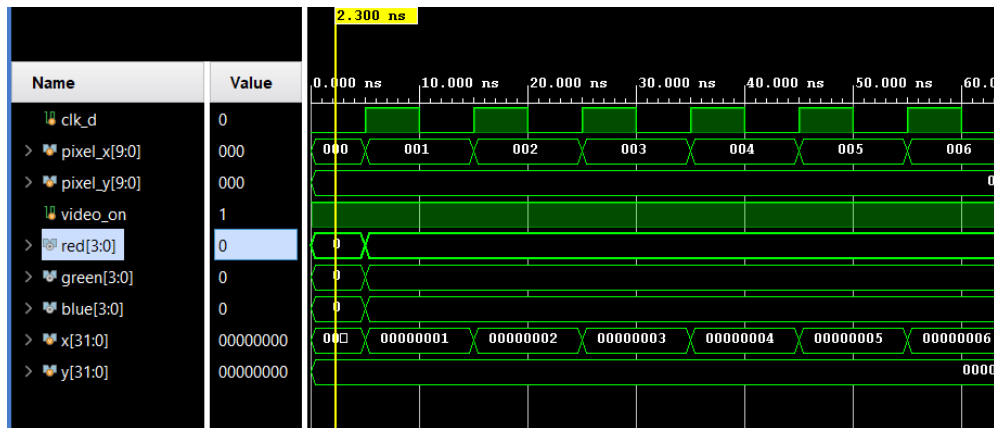
```

Results (Waveforms)

**Add snip of relevant signals' waveforms. Make sure the irrelevant area of the snip is cropped. make sure to scale properly for visibility of all case.*

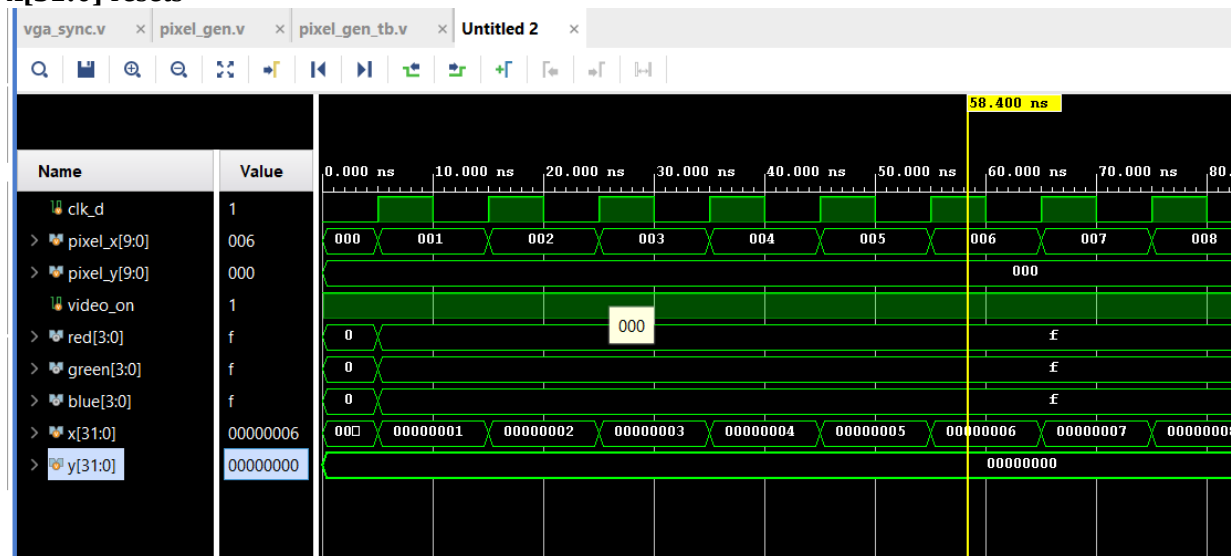
TESTCASE 1:-

- **clk_d**: This is the clock signal driving the module. You can see it oscillating between 0 and 1.
- **pixel_x[9:0]** and **pixel_y[9:0]**: These likely represent the x and y coordinates of the pixel being generated. You can observe them incrementing with each clock cycle.
- **video_on**: This signal is high (1), indicating that the video generation is active.
- **red[3:0]**, **green[3:0]**, **blue[3:0]**: These represent the color components of the pixel. In this snippet, they appear to be staying at 0, meaning the generated pixels are black.
- **x[31:0]** and **y[31:0]**: These might be internal counters or registers related to the pixel coordinates, also incrementing with the clock.



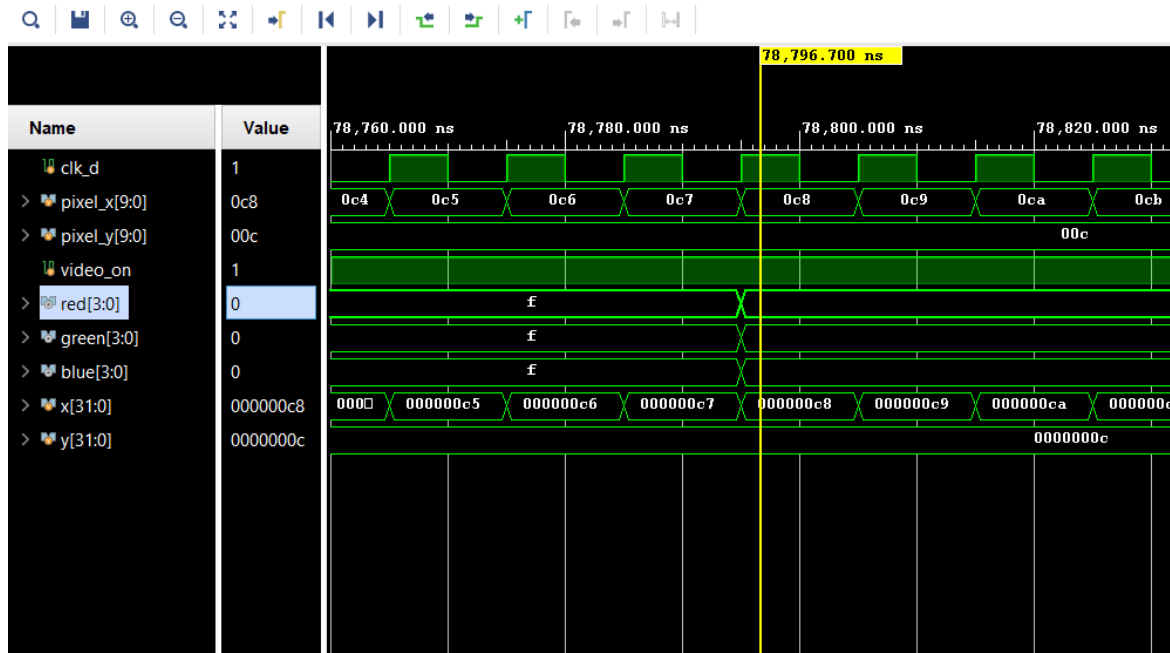
TESTCASE 2:-

- You can now see that at around 30 ns, the **red[3:0]** signal briefly becomes 000 (binary), while **green[3:0]** and **blue[3:0]** remain at f (likely representing their maximum value in hexadecimal, meaning full intensity if this is an RGB representation). Around 50 ns, all color signals (**red[3:0]**, **green[3:0]**, **blue[3:0]**) change to f. Notice that **pixel_y[9:0]** increments to 001 at approximately 60 ns, coinciding with **pixel_x[9:0]** resetting to 000. This suggests the module has completed a horizontal scan and is now moving to the next line. Correspondingly, **y[31:0]** also increments while **x[31:0]** resets



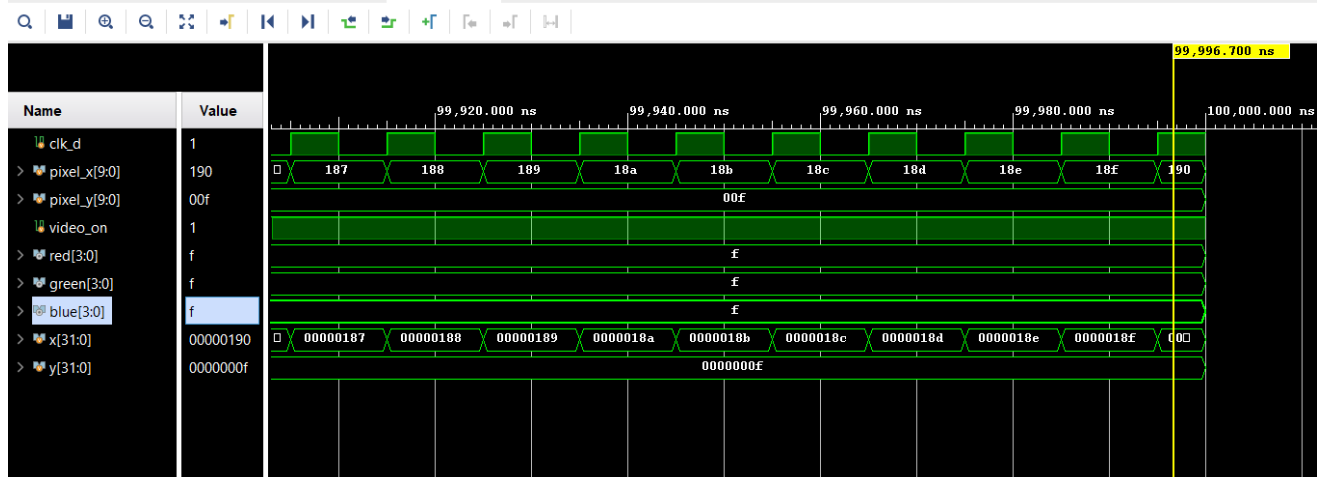
TESTCASE 3:-

The simulation is now focused around the 78,760 ns to 78,820 ns mark. **pixel_x[9:0]** continues to increment with each clock cycle, moving from 0c4 to 0cb. **pixel_y[9:0]** remains constant at 00c during this period, indicating that the module is still on the same horizontal line. **video_on** is still high, so pixel generation remains active. Notice a change in the **red[3:0]** signal. It transitions from f to 0 around 78,790 ns and then back to f shortly after 78,800 ns. The **green[3:0]** and **blue[3:0]** signals stay at f. This suggests a brief change in the red color component of the generated pixels. Correspondingly, **x[31:0]** continues to increment, and **y[31:0]** remains stable.



TESTCASE 4:-

The timeline now focuses around the 99,920 ns to 100,000 ns mark. pixel_x[9:0] continues its horizontal scan, incrementing from 187 to 190. Notice that pixel_y[9:0] has now incremented to 00f. This signifies that the module has moved to a new horizontal line after completing the previous one. video_on remains high, keeping the pixel generation active. The color signals red[3:0], green[3:0], and blue[3:0] all stay at f, indicating that the generated pixels are currently white (assuming f represents the maximum intensity for each color component). Correspondingly, x[31:0] continues to increment, and y[31:0] now reflects the change in the vertical position, showing 0000000f.



Task c: Top-Level Module/VGA Controller

Create a Top-Level Module as in Figure 11. 7. This module will receive a 100Mhz Clock as input and generate 1-bit h_sync and v_sync signals and three 4-bit signals for Red, Green and Blue color.

Code: Design module & testbench

Provide appropriately commented code for designed module & its testbench, code should contain meaningful variable names.

*Add snippet or Copy paste the code written in the Editor window. Make sure the irrelevant area of the snip is cropped.

SOURCE CODE:-

```
--
22
23 module VGA_Controller(
24     input clk,                // 100MHz clock input
25     output h_sync,            // Horizontal sync
26     output v_sync,            // Vertical sync
27     output [3:0] red,         // Red color output
28     output [3:0] green,       // Green color output
29     output [3:0] blue         // Blue color output
30 );
31
32     wire clk_5kHz;             // 5kHz clock signal
33     wire [9:0] h_count;        // Horizontal counter
34     wire [9:0] v_count;        // Vertical counter
35     wire video_on;             // Active video signal
36
37
38     // Clock divider: 100MHz to 5kHz
39     clk_div clk_div_inst (clk, clk_5kHz);
40
41     TopLevelModule_clk top_inst (clk, h_count , v_count );
42
43     // VGA sync generator
44     vga_sync vga_sync_inst (h_count, v_count, h_sync, v_sync, video_on);
45
46     // Pixel generator (Color output logic)
47     pixel_gen pixel_gen_inst (clk_5kHz, h_count, v_count, video_on, red, green, blue);
48
49 endmodule
```

TESTBENCH:-

```
`timescale 1ns / 1ps

module VGA_Controller_tb;
    reg clk;
    wire h_sync, v_sync;
    wire [3:0] red, green, blue;

    // Instantiate the Unit Under Test (UUT)
    VGA_Controller uut (
        clk,
        h_sync,
        v_sync,
        red,
        green,
        blue
    );

    // Clock generation
    initial begin
        clk = 0;
        forever #5 clk = ~clk; // 10ns clock period (100MHz)
    end

    // Simulation control
    initial begin
        #100; // Run simulation for some time
        $stop;
    end
endmodule
```

Constraint File

Copy & paste your constraint file (.xdc) here:

```
set_property PACKAGE_PIN J17 [get_ports {green[0]}]
set_property PACKAGE_PIN H17 [get_ports {green[1]}]
set_property PACKAGE_PIN G17 [get_ports {green[2]}]
set_property PACKAGE_PIN D17 [get_ports {green[3]}]
set_property IOSTANDARD LVCMOS33 [get_ports {green[*]}]

set_property PACKAGE_PIN N18 [get_ports {blue[0]}]
set_property PACKAGE_PIN L18 [get_ports {blue[1]}]
set_property PACKAGE_PIN K18 [get_ports {blue[2]}]
set_property PACKAGE_PIN J18 [get_ports {blue[3]}]
set_property IOSTANDARD LVCMOS33 [get_ports {blue[*]}]

set_property PACKAGE_PIN W5 [get_ports clk]
set_property IOSTANDARD LVCMOS33 [get_ports clk]

set_property PACKAGE_PIN P19 [get_ports h_sync]
set_property IOSTANDARD LVCMOS33 [get_ports h_sync]

set_property PACKAGE_PIN R19 [get_ports v_sync]
set_property IOSTANDARD LVCMOS33 [get_ports v_sync]

set_property PACKAGE_PIN G19 [get_ports {red[0]}]
set_property PACKAGE_PIN H19 [get_ports {red[1]}]
set_property PACKAGE_PIN J19 [get_ports {red[2]}]
set_property PACKAGE_PIN N19 [get_ports {red[3]}]
set_property IOSTANDARD LVCMOS33 [get_ports {red[*]}]
```

Results

Implement₁ the VGA controller on BASYS-03 board and show output on VGA monitor. The display output is shown in Figure 11. 8. The VGA port connections are shown in Figure 11. 9.

Demonstrate your working VGA monitor to respective RA. Attach the picture of your output screen.





Comments

**Observation/Comments on the obtained results/working of code.*

This code describes a VGA controller module. It takes a fast clock (100MHz), divides it to a slower clock (5kHz), and uses horizontal and vertical counters to generate VGA synchronization signals (`h_sync`, `v_sync`) and pixel color data (`red`, `green`, `blue`). The simulation shows the counters incrementing and the pixel color changing over time, indicating the module is scanning through pixels and generating a display signal.

Exercise

Code: Design module & testbench

Provide appropriately commented code for designed module & its testbench, code should contain meaningful variable naming.

*Add snippet or Copy paste the code written in the Editor window. Make sure the irrelevant area of the snip is cropped.

PIXEL GEN CODE FOR CHESS BOARD:-

```

1  `timescale 1ns / 1ps
2  module pixel_gen(
3      input clk_d,
4      input [9:0] pixel_x,
5      input [9:0] pixel_y,
6      input video_on,
7      output reg [3:0] red = 0,
8      output reg [3:0] green = 0,
9      output reg [3:0] blue = 0
10 );
11
12 // Parameters
13 parameter BOARD_SIZE = 480;      // Board dimensions (480x480)
14 parameter GRID_SIZE = 8;        // 8x8 board
15 parameter SQUARE_SIZE = BOARD_SIZE / GRID_SIZE; // 60 pixels per square
16
17 // Center the board on a 640x480 screen
18 localparam LEFT  = (640 - BOARD_SIZE) / 2;
19 localparam RIGHT = (640 + BOARD_SIZE) / 2;
20 localparam TOP    = (480 - BOARD_SIZE) / 2;
21 localparam BOTTOM = (480 + BOARD_SIZE) / 2;
22
23 always @(posedge clk_d) begin
24     if (video_on) begin
25         // Draw border
26         if ((pixel_x == LEFT) || (pixel_x == RIGHT - 1) ||
27             (pixel_y == TOP) || (pixel_y == BOTTOM - 1)) begin
28             red   <= 4'hF;
29             green <= 4'hF;
30             blue  <= 4'hF;
31         end
32         // Inside board area

```

```

        blue <= 4'hF;
    end
    // Inside board area
    else if ((pixel_x >= LEFT) && (pixel_x < RIGHT) &&
        (pixel_y >= TOP) && (pixel_y < BOTTOM)) begin
        // Calculate square indices
        if (((pixel_x - LEFT) / SQUARE_SIZE) % 2) ^
            ((pixel_y - TOP) / SQUARE_SIZE) % 2) begin
            red <= 4'hF;
            green <= 4'hF;
            blue <= 4'hF; // White square
        end else begin
            red <= 4'h0;
            green <= 4'h0;
            blue <= 4'h0; // Black square
        end
    end
    // Outside the board Brown color
    else begin
        red <= 4'h2;
        green <= 4'h1;
        blue <= 4'h0;
    end
end else begin
    // If video is off, output black
    red <= 4'h0;
    green <= 4'h0;
    blue <= 4'h0;
end
end
endmodule

```

TESTBENCH:-

```

`timescale 1ns / 1ps

module pixel_gen_full_screen_tb;
    reg clk_d;
    reg [9:0] pixel_x;
    reg [9:0] pixel_y;
    reg video_on;
    wire [3:0] red;
    wire [3:0] green;
    wire [3:0] blue;

    integer x, y;

    // Instantiate the Device Under Test (DUT)
    pixel_gen uut (
        .clk_d(clk_d),
        .pixel_x(pixel_x),
        .pixel_y(pixel_y),
        .video_on(video_on),
        .red(red),
        .green(green),
        .blue(blue)
    );

    // Clock generator: 10 ns period
    initial begin
        clk_d = 0;
        forever #5 clk_d = ~clk_d;
    end

    // Full-screen scan
    initial begin
        video_on = 1; // Enable video output
    end
endmodule

```



```
// Full-screen scan
)
initial begin
    video_on = 1; // Enable video output

    for (y = 0; y < 480; y = y + 1) begin
    )
        for (x = 0; x < 640; x = x + 1) begin
            // Apply pixel coordinates
            pixel_x = x;
            pixel_y = y;

            // Wait for the rising edge so that color outputs update
            @(posedge clk_d);

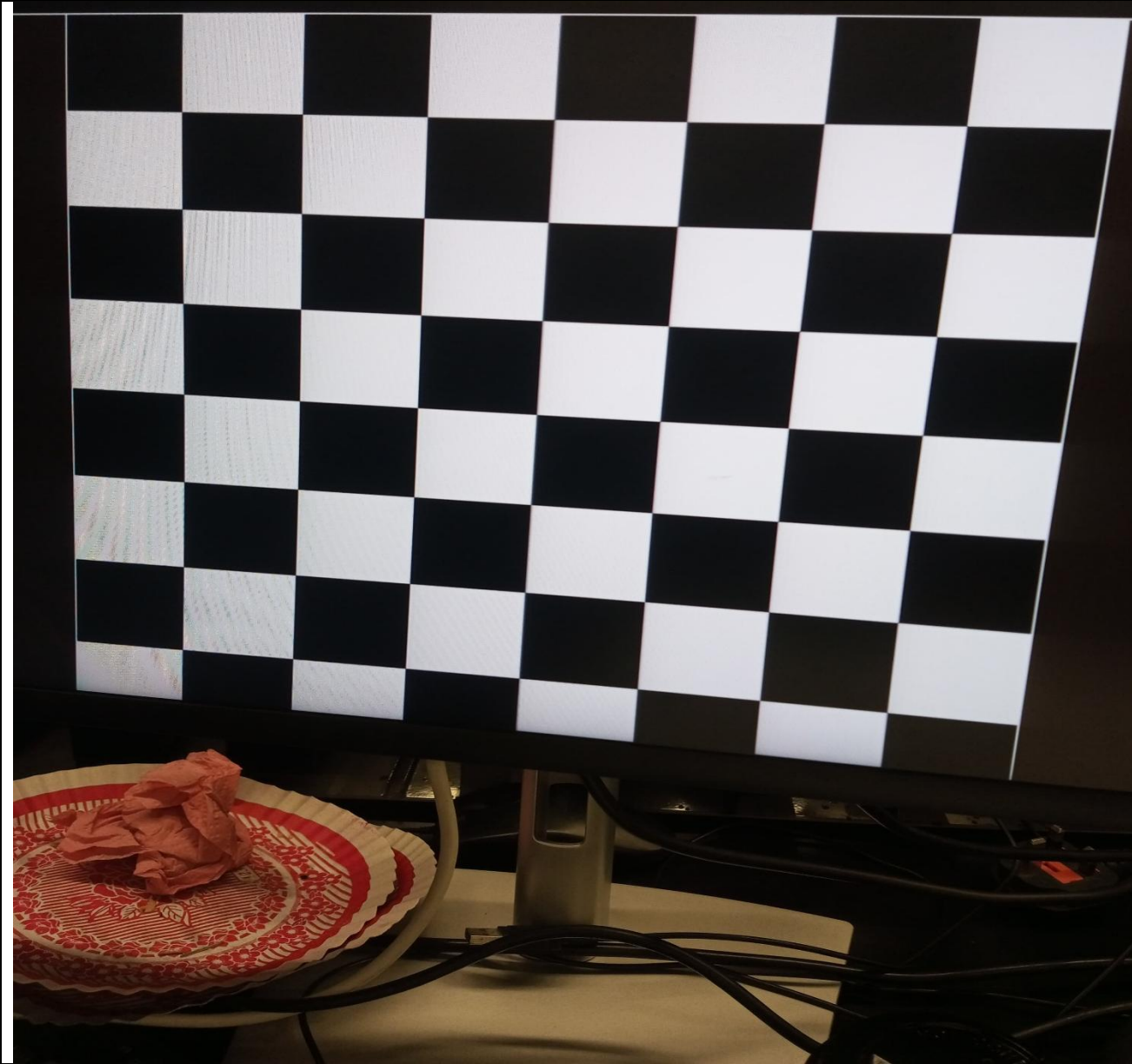
            $display("At (%0d, %0d): Red = %h, Green = %h, Blue = %h", x, y, red, green, blue);
        )
        end
    )

    // Stop the simulation after full screen test
    $stop;
)
end

) endmodule
```

Results (Waveforms)

Attach a clear picture of the Monitor output here



Comments

**Observation/Comments on the obtained results/working of code.*

- **Centering & Board Definition:** It defines a 480x480 pixel area (`BOARD_SIZE`) centered on the screen using `LEFT`, `RIGHT`, `TOP`, `BOTTOM` coordinates. This board is divided into an 8x8 grid (`GRID_SIZE`), making each square 60x60 pixels (`SQUARE_SIZE`).
- **Video On Check:** The logic only generates colors when the `video_on` signal is high. Otherwise, it outputs black.
- **Drawing Logic (when video is on):**
 - Border:** If the pixel coordinates (`pixel_x`, `pixel_y`) match the exact edges of the centered board (`LEFT`, `RIGHT-1`, `TOP`, `BOTTOM-1`), it draws a white border.
 - Checkerboard:** If the pixel is *inside* the board area (but not the border), it calculates which square the pixel falls into. Using modulo arithmetic and XOR (^), it determines if the square should be black or white, creating an alternating checkerboard pattern.
 - Outside Area:** If the pixel is outside the defined board area, it outputs a brown color (`R=2`, `G=1`, `B=0`).



Assessment Rubric

Lab 11

VGA Controller

Name:	Student ID:
-------	-------------

Marks Distribution:

	Task No.	LR2 Code	LR5 Results
In - Lab	Task a	/20	/05
	Task b	-	-
	Task c	/10	/15
Exercise		/20	/15
Total: 100		/50	/35
CLO Mapping		CLO3	CLO3

Affective Domain Rubric		Points	CLO Mapped
AR 7	Report Submission and Viva	/15	CLO 4

CLO	Total Points	Points Obtained
3	85	
4	15	
Total	100	

For description of different levels of the mapped rubrics, please refer to the provided Lab Evaluation Assessment Rubrics and Affective Domain Assessment Rubrics.



Lab Evaluation Assessment Rubrics

#	Assessment Elements	Level 1: Unsatisfactory Points 0-1	Level 2: Developing Points 2	Level 3: Good Points 3	Level 4: Exemplary Points 4
LR1	Circuit Layout	Connections between circuit components are mostly wrong. Circuit layout is cluttered. Needs guidance to make correct equipment/ component connections.	Few of the connections made between circuit components are wrong. Circuit layout is not neat and clean as per standards.	Circuit layout and connections are correct but not all connections are neat and clean as per standards.	Neat, clean and correct connections of all the circuit components/ equipment are made as per standard circuit diagram.
LR3	Troubleshooting	Unable to identify the fault/minimal effort shown in troubleshooting.	Able to identify the fault but unable to remove it.	Able to identify the fault but partially removes it.	Able to identify the fault and takes necessary steps and actions to correct it.
LR5	Results & Plots	Figures/ graphs / tables are not developed or are poorly constructed with erroneous results. Titles, captions, units are not mentioned. Data is presented in an obscure manner.	Figures, graphs and tables are drawn but contain errors. Titles, captions, units are not accurate. Data presentation is not too clear.	All figures, graphs, tables are correctly drawn but contain minor errors or some of the details are missing.	Figures / graphs / tables are correctly drawn and appropriate titles/captions and proper units are mentioned. Data presentation is systematic.
LR7	Viva	Response shows a complete lack of understanding of the assigned / completed task	Response shows shallow understanding of the assigned task	Response shows substantial understanding of the assigned task	Response shows complete understanding of the completed task. The student is able to explain all the related concepts.
LR10	Analysis	Results are not interpreted or are analyzed incorrectly.	Analysis of the obtained results is incomplete. Conclusions are not drawn.	Results are analyzed and concluded but are not well explained and justification is not provided with the help of reasoning.	Results are well analyzed supported with reasons. Good comparison is made between the theoretical and experimental results with correct and useful conclusion.
LR11	Design	Proposed design is unsubstantiated or does not satisfy most of given constraints. The breakdown of system into features is incomplete and still at lower resolution.	Justification for proposed design is weak, and it only satisfies some constraints. The breakdown of system is missing some features and resolution is not appropriate at some places.	Proposed design is substantiated, preferably through proper analysis, and satisfies most given constraints. The breakdown of system into features seems complete, but resolution is not appropriate at some places.	Proposed design is substantiated, preferably through proper analysis, and satisfies all given constraints. The breakdown of system into features seems complete and at appropriate resolution, given students' level.